UPPSALA UNIVERSITET

UPTEC STS 24043
Examensarbete 30 hp
September 2024

# Text Summarization using a Transformer Architecture

## An Attention based Transformer approach to Abstractive Summarization

Jonas Jons

Civilingenjör i System i Teknik och Samhälle

UPPSALA UNIVERSITET

Text Summarization using a Transformer Architecture

Jonas Jons


## Abstract

With an ever-growing volume of data on the internet and in literature, grasping the full picture of a subject becomes increasingly difficult. One such area that has been heavily studied is the COVID-19 pandemic, which prompted a global scientific race to stop, mitigate, and protect against the virus.

To help manage the vast number of studies on COVID-19 and conclude the scientific findings, the White House launched the CORD-19 Dataset, an open-source project compiling many of these studies. Hosted on the popular data science community website Kaggle, this project called for the community's aid in deriving new insights from the extensive research available. This study focuses on two main subjects, with inspiration from the aforementioned subjects: the ever-growing amount of data and the CORD-19 open-source project.

Grounded in the influential paper “Attention is All You Need” created in 2017, which introduced the original transformer model (now used in applications like ChatGPT), this study aims to create a transformer model from scratch to perform text summarization on the samples in the CORD-19 dataset. By combining cutting-edge transformer technology with the analysis of the CORD-19 dataset, the study provides valuable contributions to both areas. This is especially important as the scientific literature on transformers is currently limited, given the recent development of this type of deep learning network.

In this thesis, the data from the CORD-19 dataset is downloaded, cleaned, and processed. It is then fed into a custom-built transformer neural network, specifically modified for the task of text summarization. Building the network from scratch, rather than using a pre-built model, aims to foster a deeper understanding of the fundamental technology and its mechanics. The conclusions and results of this thesis will offer valuable insights into both the transformer model and the challenges associated with analyzing the CORD-19 dataset.



**Teknisk-naturvetenskapliga fakulteten**

**Uppsala universitet, Utgivningsort Uppsala/Visby**

Handledare: Mikael Axelsson (Consid Uppsala AB) Ämnesgranskare: Ingela Nyström

Examinator: Elísabet Andrésdóttir

# Populärvetenskaplig Sammanfattning

Andelen data och information på internet växer årligen exponentiellt, och förväntas enbart fortsätta i samma stil genom åren framöver (Taylor 2023). Mängden data som finns idag inom diverse ämnen är så enorma att det är otroligt svårt för en individ att ens vara påläst om all information inom en specialiserad disciplin. Det kan finnas tusentals artiklar spridda över flertalet olika språk, vilket gör att för att få en väl grundad förståelse för något, kan informationen som behöver granskas upplevas orimlig.

Ett av dessa områden som har varit i världens fokus de senaste åren är den av Corona 2019 pandemin, vilket ledde till kapprustning inom den akademiska världen för att motverka dess harmfulla och dödliga effekter på samhället. Forskare världen över investerade sig djupt i att motarbeta viruset och detta ledde till att informationen och forskningen inom ämnet snabbt växte. För att etablera en djupare förståelse av de studier och data man insamlat etablerade det amerikanska vita huset ett open-source projekt benämnt CORD-19 som var en sammanställning av uppemot 401 000 artiklar på ämnet av corona pandemin. Detta projekt publicerades i samarbete med Machine learning / Artificial intelligence community:n Kaggle och ämnade att inspirera folk till att bidra genom att utföra analyser av datasetet för att bättre förstå den information som har insamlats.

Denna studie ämnar att bidra till arbetet på CORD-19 projektet genom att bearbeta datan och identifiera problem med den, samt applicera en neural nätverkstyp benämnd `'Transformer" för att utföra abstraktiv text-summering, vilket innebär att nätverket genererar en summering utifrån given kontext, där nätverket själv identifierar de viktigaste aspekterna och skriver summeringen. Denna studie ämnar således att kombinera flertalet aktivt studerade områden såsom transformers och CORD-19 datasetet, medans den samtidigt ger ett perspektiv på hur summering som koncept byggs och implementeras, för att förfina mängden data till ett mer brukbart och lättläsligare sätt.

# Acknowledgements

This master's thesis project was developed at Uppsala University in collaboration with Consid AB:s office in Uppsala. I would like to extend my thanks to my supervisors at Consid, being Erik Löthman and Mikael Axelsson, and thank them for their aid in formalizing and planning the thesis project as well as my subject reviewer Ingela Nyström for her valuable help in constructing the report and aiding with academic insight and support as well as pushing me on to finalize the project. Without Ingelas support there was a decent possibility that the project would never have been finalized due to its original complexity of creating, building and training a transformer neural network architecture from scratch, something that after a while seemed too complex for a study of this scope. But, her perseverance motivated me to continue and finalize the project, for which I am very grateful as I have learned more than what can be described in this study and acquired the skill to work with today's most cutting edge technology within the field of natural language processing. Experiences and teachings I will forever have with me.

**1. Introduction....1**
1.1 Problem Definition.... 2
1.2 Limitations of the Study.... 2
1.3 Thesis Disposition....3
**2. Background....5**
2.1 Machine Learning.... 5
2.2 Neural Networks.... 7
2.3 Deep Learning.... 8
2.4 Recurrent Neural Networks (RNN).... 8
2.5 Tokenization....10
2.6 Attention models.... 11
2.6.1 Scaled Dot Product Attention....12
2.6.2 Multi Headed Attention....13
2.6.3 Multi Headed Cross Attention.... 14
2.6.4 Masking....14
2.7 Transformers....14
2.7.1 Shared Qualities of the Encoder and Decoder Blocks....15
2.7.2 The Encoder Block....17
2.7.3 The Decoder Block.... 18
2.7.4 The full Transformer Network in “Attention is all you need”....20
2.8 Variations of Artificially Generated Text Summaries....20
2.9 Related Work....21
**3. Method....23**
3.1 Why Abstract to Title summarization.... 23
3.2 From Extractive to Abstractive summarization.... 23
3.3 High level explanation of the Model and the Data.... 24
3.4 Why Transformer model instead of RNN?....24
3.5 Hardware, Design tools and Development environment.... 25
3.6 Dataloader, Optimizer and Loss Function....25
3.7 Model parameters....25
3.8 Regarding choice of parameters....26
3.8.1 Learning rate....26
3.8.2 Hyperparameters: d_model, num_heads and num_layers....27
3.8.3 Total amount of parameters....27
**4. Data....31**
4.1 Dataset: COVID-19 Open Research Dataset (CORD-19)....31
4.2 Data Preparation....31
4.2.1 Data preprocessing and the parameters.... 31
4.2.2 Showcase of exemplary data before tokenization.... 33
4.2.3 Data outliers....33
4.2.4 Tokenization of the data....34
4.2.5 Batching....35
**5. Results....37**

5.1 Qualitative results: Samples..... 37
5.1.1 Three AI Generated summaries and target Abstracts..... 37
5.2 Quantitative Results: Loss calculations..... 38
5.2.1 Loss calculation: Training set.....38
5.2.2 Loss calculation: Validation set..... 39
**6. Discussion.....41**
6.1 Performance comparison to actual titles.....41
6.2 Critical Discussion of the data..... 42
6.3 Critical Discussion of the model.....43
6.4 Ethical Aspects..... 44
6.5 Possible improvements.....44
**7. Closing Remarks..... 47**
7.1 Conclusions..... 47
7.2 Educational Highlights..... 48
7.3 Future work.....49
**8. References..... 51**

# 1. Introduction

Artificial Intelligence (AI), Machine Learning (ML) and Deep Learning (DL) have become the buzzwords of the 21st century. With the recent development and publication of ChatGPT, AI is no longer limited to discussions within computer science disciplines and movies, it has revolutionized the public's view of AI. Behind the concept of ChatGPT are many highly advanced technologies, but at its core is what's referred to as a Transformer model (GPT - Generative Pretrained Transformer). A transformer is a form of deep learning that was recently invented by a group of scientists and published in the now famous paper “Attention is all you need” in 2017, and is at the core of this thesis focus. (Vaswani et al., 2017).

So what is really a transformer in terms of computer science and deep learning and how come it brought about such a swing in our entire society? To fully grasp the complexity and impact of the transformer it is important to first understand the field of Natural Language Processing (NLP) and what problems are defined within the field. NLP is an interdisciplinary subfield of linguistics and computer science that searches to unravel the understanding of languages and translate it into mathematics and modeling. A means to artificially generate, understand and process languages. (IBM, n.d.)

Before the arrival of transformers, Recurrent Neural Networks (RNN) was the most common form of deep learning for NLP tasks, working with languages and text interpretation as it specialized in handling sequences of data, but it came with several flaws. Amongst others, as data became longer and longer the complexity of sequences became all the more problematic and the model suffered performance issues, there are also major issues in utilizing parallel computation as the input sequences into a RNN model has to be processed sequentially, meaning training is really slow and inefficient. Transformers improved upon these issues and revolutionized the field, evolving the way machines and people could interact, first and foremost in text (Culurciello 2018).

With an exponential growth of data on the internet, obtaining a generalized perspective of information on topics has become more complex and there may be thousands of answers to a particular question, all formulated in different ways. One of these widely discussed topics of today is the Corona Pandemic that initially started its spread in late 2019, as the data and research on the subject grew a global initiative was announced on part of the White House to get a better grasp of all the data that was produced on the topic of corona and employ open source development on the dataset to contribute to the efforts of battling the virus. This thesis aims to contribute to this effort by working with and employing the data for usage in a transformer based model for text summarization, whilst also exploring and identifying problematic issues for large scale implementation of the CORD-dataset, whilst also contributing further to the current research on the topic of building and training transformer networks (Allen Institute for AI, 2020).

## 1.1 Problem Definition

This study aims to apply a deep learning transformer model to perform abstractive text summarization on the papers found in the CORD-19 dataset. The generated samples from the model summaries will be compared to some of the papers original samples to perform a qualitative assessment of the results. The study will also investigate and build the transformer model architecture from scratch, to achieve a deeper understanding of its capabilities and contribute with the study's findings to the scientific literature on the subject.

Therefore, the study aims to answer the following research questions:

- How well does a Transformer model trained and built from scratch perform at the task of abstractive text summarization on the CORD-19 dataset?
- What teachings can be derived from building a transformer model from scratch?
- How can this study's research be built upon and what areas of possible improvements are there with regards to the dataset and the model?

## 1.2 Limitations of the Study

Due to time restrictions and limited computational resources, the study chose to perform the task of summarization on the abstracts, rather than the larger body chunks of text for the papers in the CORD-19 dataset. As will be showcased in the latter sections, more specifically Section 4 regarding the data, many of the papers have a large amount of content (sometimes abstracts in the thousands of words), and far larger actual texts. This makes it initially too challenging for a transformer model to train from scratch on, and even proves difficult for some of the OpenAI models of today like chatGPT which has a limit of around 6000 words as context window and has trained with more resources and data than this study can possibly achieve.

This might seem a bit contradictory to the task of summarization, but in actuality would be an important stepping stone for building a model for achieving these larger summations regardless. If a model is trained to summarize smaller contexts, a hierarchical text summarization process could be done to chunk up the textbody into separate sequences, summarizing them step by step and then creating a finalized summary. This implies that whether the target is creating an abstract from the text body, or creating a title from the abstract, it's still a summarization process where the model can be finetuned and worked upon further to tackle larger chunks of text. This limitation is further developed upon section 3.1 as it is found to be an important subject for the study's purpose, especially in relation to the CORD-19 dataset.

Another limitation employed by the study is that only papers in the English language will be evaluated as introducing other languages enlarges the task by quite a lot of parameters which are beyond the scope achievable by this study. The CORD-19 dataset does involve some samples of other languages, but too few in samples to prove of any actual value to the model, and instead only provides noise for the purposes of training and evaluation.

Furthermore, only a selection of the data from the CORD-19 dataset is used, as there is a large variance of the samples found in the data. Abstracts for example can range from around 5 words to upwards of the several thousands, and some titles can even be upwards of hundreds or low thousands, which may be considered wrongful data or outliers. Regardless, for training the model on somewhat uniform data, a narrower scope of lengths were chosen as samples from the dataset, this is also further developed upon in Section 3 and Section 4.

## 1.3 Thesis Disposition

The thesis is divided into six primary sections: Background, Method, Data, Results, Discussion and Closing Remarks. The background section will examine the theoretical frameworks used in the study and introduce you as the reader to the concepts of Deep Learning, RNN:s and the transformer model. This section will give you as the reader a better understanding of how the model has been built and make it easier to follow along in the coming sections where otherwise unfamiliar concepts might be used frequently. The following section thereafter, namely Method, aims to detail how the results and preparatory work was conducted and present any assumptions or decisions made in construction of the model.

Data will give a more thorough insight into how the dataset is composed and key attributes accessed and used for inputting the data into the model. Results will display the concluded product of the models training process and performance in accordance with the performance measurements. These results will then be discussed and further developed in the Discussion section, thereafter a conclusion will be presented with regards to the problem definition in section 1.1.

# 2. Background

This section of the thesis presents the reader with the concepts necessary to understand the studies theoretical aspects and concepts. Lighter introduction to the concepts of ML/AI/DL will be presented along with a deeper dive into the concepts creating the foundation of the thesis, being the Transformer model and the concept of Attention which is a fundamental component of  the underlying architecture for the transformer models.

## 2.1 Machine Learning

Machine Learning is a field of computer science where an algorithm that is trained on data attains the ability to identify patterns and optionally draw conclusions from the data (Machine learning, 2023). A classic example would be that of identifying if there is a cat in a picture or not. For this purpose, there are several types of algorithms that can achieve this task, but the simplest algorithm would be that of logistic regression. Logistic regression can be illustrated as a very simple form of a neural network with just one layer and one activation function, where the input is a picture of a cat converted to 12288 data points. The number 12288 is derived from the preprocessing phase of converting the original image to a 64x64 pixel image, where each pixel is a combination of the three different colors Red, Green and Blue (RGB). Thus, each pixel has one value for each of the colors, which is equivalent to a total combination of 64*64*3, which equates to 12288 input data points for each picture of a cat. In the below example a theoretical model predicted the image with a 79% certainty to have a cat in it. This would be under the assumption that our model predicts highly for cat pictures and low where there is no cat present. If the algorithm outputs a value higher than 0.5, we conclude that there is a cat in the picture we are looking at, if below 0.5 the network is instructed to identify the picture as not containing a cat.  The terminology of neural networks and what an activation function is will be further discussed under section 2.2. A visualization of this simplistic theoretical neural network can be found in Figure 1 (Logistic regression, 2023).

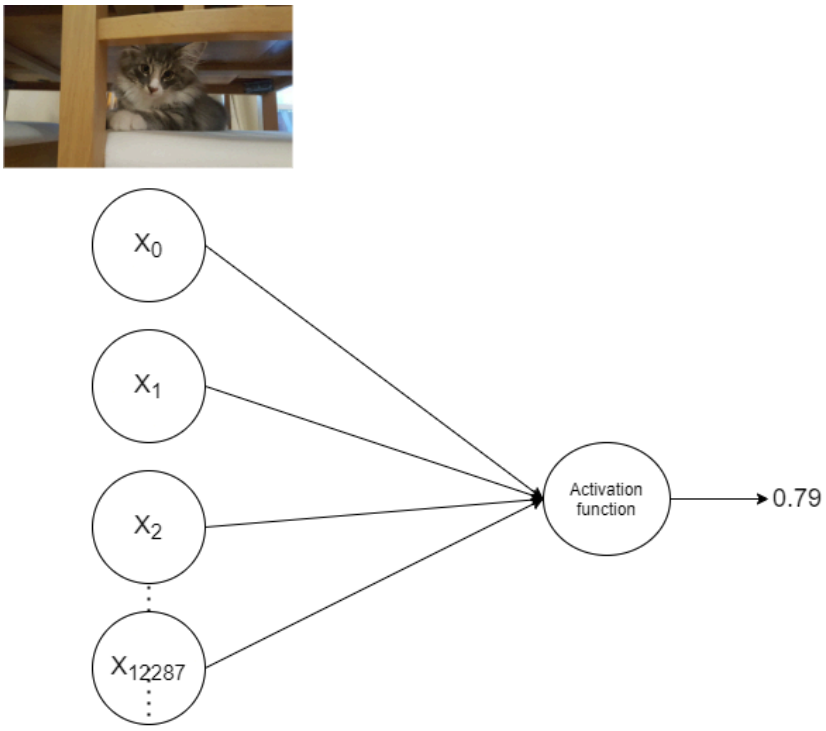


Figure 1: Cat identifier using logistic regression.

One of the issues machine learning faces is that of computational expense, meaning the cost of training an algorithm or model to acceptable performance might drain a lot of resources, as many computations need to be made to achieve the process of learning. The illustration in Figure 1 shows the input for a single example of a cat picture for a model, to create a somewhat acceptable accuracy for a trained model it could be hypothesized that there would need to be thousands if not tens of thousands of examples or more needed for the training. How well the algorithm learns is highly dependent on several factors such as the data, hyperparameters and the structure of the network. When the classification problems grow in complexity, a model such as this becomes ineffective, and the model or the hyperparameters or both need to be adjusted to handle increasing difficulty of the task. An example of this would be identifying how many cats there are in a picture, a task this network is unable to accomplish as it effectively only answers “yes” or “no”. With increasing complexity of problems to solve for a network, the  higher the needs for complexity in the model, the higher the need of computational resources.

In machine learning tasks the type of data used is often categorized into one of two categories, either supervised or unsupervised. Supervised machine learning is a subarea of machine learning where the algorithm learns to understand the data by being fed labeled examples, if sticking with the example from above it could be for example that the algorithm learns on pictures that are labeled with either y = 1 (picture contains a cat), or y = 0 (picture does not contain a cat). This then tells the algorithm what the correct prediction should be and then adjusts the hyperparameters to better suit the predictions. This is what is called Supervised machine learning, its counterpart being

unsupervised machine learning where the model must find patterns by itself. This study will focus on training a supervised machine learning model on articles where the data contains both the abstract as well as a handwritten title by the authors. The title for each paper's model is the “correct” label, or the “target” for the model.

## 2.2 Neural Networks

An example of a simple neural network (also referred to as shallow neural network) was detailed in the previous section with the cat image example. In Figure 2 a neural network handling the same task as the logistic regression problem is detailed, but with additional layers. A network model such as the ones found in Figure 1 and 2 function through a mechanism referred to as a feed-forward process, this means that each sample of data is considered separately and the model learns from the findings of each sample but then the actual sample is forgotten about, apart from what the model learnt. This renders this standard type of network bad at interpreting the importance of sequence-to-sequence data, for example text or music analysis, as what comes before might influence what comes thereafter.

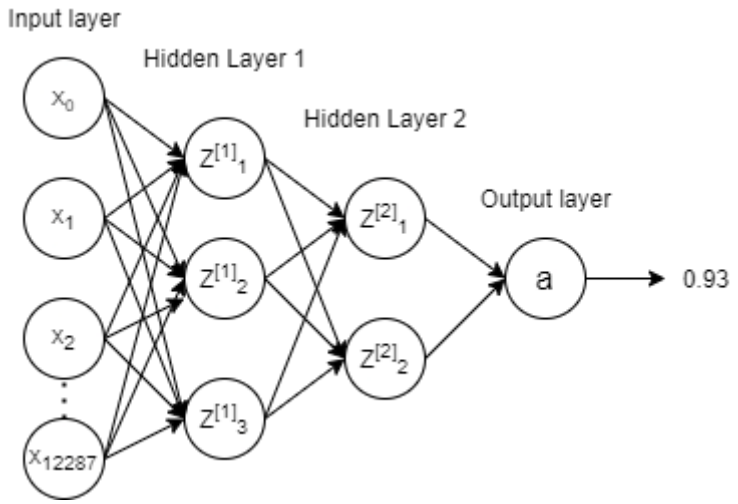


Figure 2: Cat identifier using a neural network with three layers.

The main difference between the networks in Figure 1 and Figure 2 is that the number of layers differ between the two. The neural network model in Figure 2 consists of an input layer, two hidden layers and an output layer. The input layer is as described in section 2.1 the pixel values of the image with each value corresponding to one of the x values, these are then fed to each neuron in the first hidden layer where the information is processed and then put into separate activation functions, typically these neurons are all randomly initialized which prevents the neurons in the same layer to pick up on the exact same patterns (Yadav 2018). To exemplify the notations the first neuron in the hidden layer is $Z^{[1]}_1$ this means the first neuron in the first layer, the bracket indicating which layer the neuron exists in. These neurons then in turn pass their information into the next hidden layer which performs a similar task and then finally passes it to the

output layer. The output layer then gives a final prediction regarding if a cat is in it or not. This model has more layers and is more well suited for the task of classification than the logistic regression model, hypothetically it would outperform the previous model as it is able to encapture more complex characteristics through each neuron. There are though many more parameters than just the amounts of layers that affect the effectiveness of a neural network such as task and data which is why a definitive answer can not be given in performance comparison between the networks in Figure 1 and Figure 2, but one could for example hope that this model could pick up more complex patterns of what makes a cat a cat.

## 2.3 Deep Learning

Deep Learning is an extension of the concepts previously discussed in sections 2.1 and 2.2. Mainly, the word deep learning refers to “deeper” neural networks, that is networks with a larger number of layers. There are however plenty of variants of deep neural networks and some of the most common architectures are that of the Recurrent Neural Networks (RNN) and Convolutional Neural Networks (CNN). Some common use cases for the different types are natural language processing tasks for RNN: s and image recognition and object detection for CNN: s (Deep Learning, 2023). For this thesis purpose only the theoretical framework for that of the RNN will be discussed in the following section as it has been used in related studies about text summarization and is a predecessor to the concept of the Transformer which will be developed upon in section 2.7.

## 2.4 Recurrent Neural Networks (RNN)

Recurrent Neural Networks are a subclass of Neural Networks which is specialized towards understanding and interpreting sequences of data (often referred to as Sequence-to-Sequence learning), tackling one of the issues of the feedforward process earlier discussed. The most applicable and common usage of this type of network is on problems concerning Natural Language Processing where previous data can be critical towards evaluating the data currently examined. Two good examples of this are training a neural network to recognize patterns in sentences and/or generation of a new set of sentences, with each being coherent with the other (Zhu & Chollet, 2023).

A common analogy to describe this process of memorizing and taking into consideration what comes before and after is comparing it into the human brain and how we remember events or react to things in our environment. Having once been scratched by a cat, you may next time be more cautious about approaching one on the street. It turns out that when looking at these types of systems where both the data from before and after are crucial, a typical feed-forward network such as the common deep learning models is not quite sufficient (Amidi & Amidi, n.d.). There are several versions of the RNN architecture, but two samples are displayed in Figure 3 and 4 below.

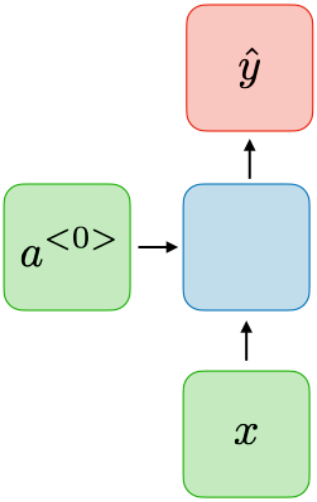


Figure 3: Recurrent neural network with one-to-one architecture (Amidi & Amidi, n.d.)

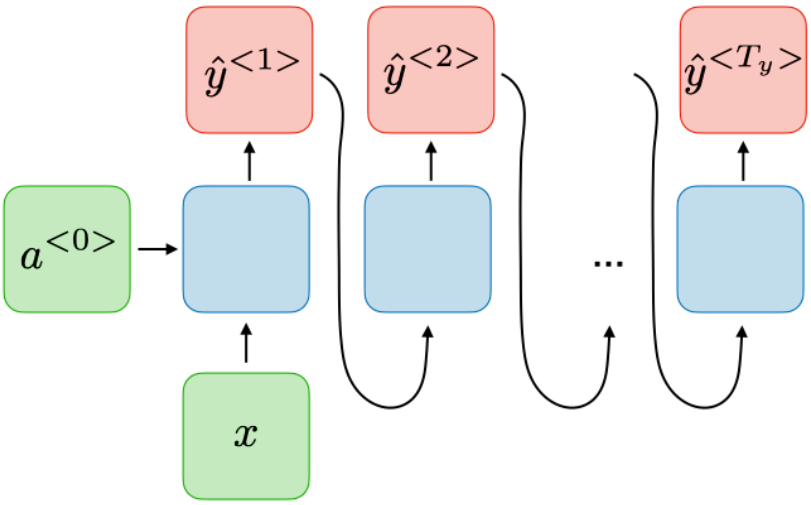


Figure 4: Recurrent Neural Network with one-to-many architecture (Amidi & Amidi, n.d.)

The network architecture found in Figure 3 resembles the most simplistic form of a RNN and is closely associated with a standard neural network, with the difference being of an initial activation function which affects the output. In this one-to-one architecture there is no real concern for what comes before or after, simply to evaluate the targeted output.

The other network in Figure 4 is of a one-to-many architecture. Conceptually what separates these is how the output from one time step becomes the input in the next timestep (here a timestep refers to the linear progression of the neural networks architecture, you could say a prediction is made at each timestep t, which then affects the predictions at timestep t+1). This in turn means that the outputs after the initial timestep are dependent on each other. This type of architecture is well applicable to tasks such as music generation where the input could be a genre and the output is a set of notes. How the notes line up and their order of appearance determines how the sound comes out and how defining it is in the genre. Thus, to determine the overall theme of the music, all the previous notes have a high impact on the conclusion.

RNN:s has seen a lot of use within NLP and has been improved upon with several additions into the architecture. Some of the more recent mechanisms typically implemented are that of Long Short-Term Memory (LSTM) or Gated Recurrent Unit (GRU). Both additions address one of the larger issues with the traditional RNN architecture, being that of preventing vanishing or exploding gradient problems (Amidi & Amidi, n.d.). These can occur whilst training the model as some sequences of input might be very large, which causes the network exponentially to train its parameters, resulting in a highly inaccurate model in most scenarios. As the input into the next timestep t is also dependent on the results in timestep t-1, the RNN models also suffer from not being able to perform parallel computations, making the learning process quite slow and inefficient for training larger models.

## 2.5 Tokenization

Tokenization is a crucial process for the transformer and most text based sequence to sequence models, including but not limited to the RNN architecture in 2.4. Tokenization represents a way of converting words into digit representations as the words in themselves are not readable by a computer. Whilst this might seem simple enough conceptually, it is in actuality a lot more complicated than one might think.

There are many ways of doing tokenization, the simplest being character level tokenization which is to convert all characters to a number. This would net a low vocabulary size of for example 29 for the Swedish language, but would come with the downside of that to generate even a singular word like “hello” you would need to generate 5 unique characters, and the model has to understand the relation between each character. For this reason, many of the best-performing tokenizers are tailor built to find this optimal point between where do we retain high value (i.e we don't have to generate that many tokens per word), but still have a lower vocabulary size netting a higher performance of the associated transformer model. Further developing on this, the word “hello” might have many different interpretations depending on the context it is used in, or how it is said, which introduces us to the second important concept that will be referenced a lot in this study, namely embedding dimensions, often denominated d_model or n_embed.

Why this is relevant for the tokenizer and the final utilization in the transformer is that a large vocabulary is detrimental because each token in the vocabulary is a product of matrix multiplication between embedding dimensions and the vocabulary size. Say for example that the embedding layer is set to contain 512 dimensions for each token in the vocabulary, that would mean that for a vocabulary of size 30000 you would have 30000 * 512 parameters as each token in the vocabulary gets 512 dimensions of "understanding" that the transformer can learn from for each token. Over the course of training which might be days or even months for the largest models, even 100 more or less items in vocabulary could contribute massively to the performance, both in terms of computational cost but also performance. With this in mind, building a well equipped tokenizer is usually a large task in and of itself. One could utilize a pre-trained and pre-built tokenizer, but it comes with both downsides and upsides. The largest downside most likely being that tokenizers often are related to their respective transformer models architecture, meaning that how the tokens are used might not be directly translatable to other transformer models, which might cause negative effects on performance. For this reason a very simple word based tokenizer is used in this study, as the focus of the study is the dataset, summarization and the transformer model itself, not building a sophisticated tokenizer. Further details upon the study's tokenizer can be found in Section 4.2.4 (Hugging Face, n.d.).

## 2.6 Attention models

In the concept of sequence-to-sequence tasks such as translating a sentence in one language into another, or text summarization, attention is a mechanism that allows for a model to pay extra *attention* towards specific words in a sequence. During training, the model learns the relations between words using attention, and what piece of the sentence yields the most information about the context. Together with embeddings, attention mechanisms enable the model to understand the full context and also the dimensions of each token, enabling the model to understand the relations and linguistic principles.

In attention the model uses three sets of vectors which are derived from the input; queries, keys and values. The queries represent the current focus of the model, which part of the input sequence we are looking for more information for. The keys represent the relevant parts of the input for the queries, meaning it gives the queries the possibilities of words to attend to and their impact on the query. The product is then normalized and multiplied by the values vector, creating the finalized product of the relation and attention between the words in the sequence.

$$Softmax\ (QK^{T)} * V$$

Equation 1: Attention formula for Dot-Product Attention

However, there are different versions of Attention that can be utilized, such as scaled dot product attention.

### 2.6.1 Scaled Dot Product Attention

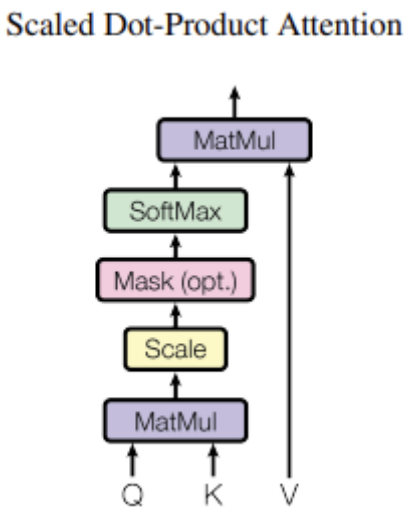


Figure 5: Scaled Dot-Product Attention (Vaswani et al. 2017)

Scaled dot product attention is an extension of the original formula where the difference lies in the denominator being the square root of the dimensionality of the key vector. Whilst without any "heads" which are introduced in the next section, this would be the same as d_model, but when using heads the d_model is split up according to the amount of heads. The inclusion of this denominator is useful in tackling the issues the original dot product attention has with scaling. For very large values of Q and K the product after softmax can lead to very small or very large gradients after softmax calculation, which can slow down learning significantly. The division by dimensionality addresses this issue and the full scaled dot product attention can be found in Equation 2.

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Equation 2: Scaled Dot Product Attention (Vaswani et al. 2017)

### 2.6.2 Multi Headed Attention

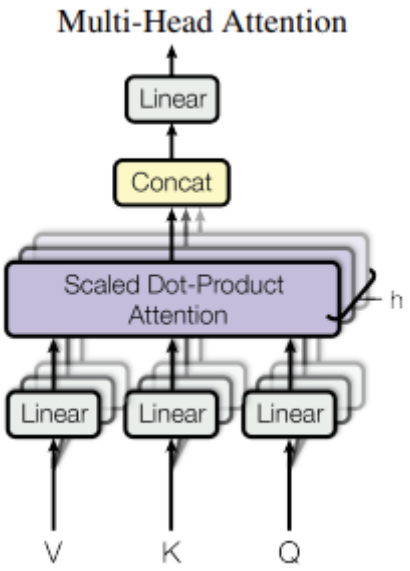


Figure 6: Multi-head Attention (Vaswani et al. 2017)

Multiheaded attention expands upon the previous concepts by devising the entire sequence into separate heads that work in parallel, this allows for each head to represent a dimension of understanding on the input sequence, intuitively allowing for the model to pay greater detail to a specific aspect in each head. A reference could be made that trying to understand all interpretations of a sentence at once, akin to answering “what are all different interpretations of this”, would be a harder task then “is it a positive sentence” or “who is it referring to”. Each head can be thought of to specialize on a subset of dimensions, allowing greater detail to be captured. The results from all the heads that work in parallel are then concatenated, yielding a more well rounded understanding of the sequence.

### 2.6.3 Multi Headed Cross Attention

There is another variant of attention than the ones referred to above, whilst the concept itself remains the same. The examples in 2.6.1 and 2.6.2 are self-attention based models, meaning that they work on the same input for keys, queries and values. An example of self attention could be paying attention to a word in a specific sentence, with its relation to the other words. An easy example of self attention:

"Hello how are you"

In this instance, attention could be calculated with a query being a particular word as for example "how" and the keys being the representation of the other words in the sequence. However, there might be other ways that the attention needs to be applied. An example of this would be the task at hand of the study, which is to perform summarization. In this scenario to generate a title, one would need to pay attention to both the input, being the abstract, but also the title that is currently generated (inference) or the actual title (during training).

This is where cross attention comes into play. In this scenario the mapping of queries keys and values are a bit different, with the keys and values coming from the input sequence (as for example the abstract) whilst the queries vector is the abstract. This allows for the model to pay attention to what is found in the input, whilst being in a different context than the target (title).

### 2.6.4 Masking

Masking is a concept of hiding items in an input sequence from the attention model. Sometimes, there might be words that are just filler and have no meaning, to reach for example a uniform sequence length across the data. In this case, special tokens may be used for this purpose, which are in this study referred to as padding tokens. This has no inherent value and effect on the input, therefore a "mask" can be applied to these tokens in the input and target, preventing attention from functioning on these parts. How this is utilized in developed upon in the following section regarding transformers.

## 2.7 Transformers

A transformer is an attention based model that helps solve or mitigate some of the issues that come with text handling for RNN:s discussed in Section 2.4. The previously discussed issues were that of problems implementing parallel computing, loss of information and the vanishing gradient issue. (Giacaglia 2019)

Akin to its RNN counterpart it is typically referred to as a Sequence to Sequence model (Seq2Seq) meaning its purpose is often to both understand the context of what it is reading, and derive a new sequence of understanding from it. This could be for instance

the translation of languages (German to Swedish) or the task of summarization (a body of text into a summary). This process usually involves a Encoder and Decoder architecture where the Encoder processes the input data (for example the abstract), and the decoder which process the target data together with the input data (for example generating a title) (Sutskever, Vinalys, and Le 2014). This allows the model to learn the relationships between the input and the target, and to generalize the patterns involved for generation of similar new contexts.

In sections 2.7.1 and 2.7.2 the architecture of the encoder and decoder networks from the paper "Attention is all you need" will be discussed, as it is the primary source of inspiration for the transformer network utilized in this study for the task of text summarization. (Vaswani et al. 2017)

Please note that in the following sections I use "target" and "title" interchangeably but they indicate the same. Target is more to generalize, and title is used to relate to the purpose of the study. Abstracts are more akin to the input. The entire transformer model can be found in Figure 10 for references to the terminologies used in section 2.7.1.

### 2.7.1 Shared Qualities of the Encoder and Decoder Blocks

The encoder and decoder blocks as can be seen in figure 8 and 9 respectively contain some similar components, in fact, they are very much alike with the exception of the case that the decoder takes the output of the encoder block as its input in the second stage of multi headed attention which is cross attention. As multi-headed attention has already been discussed in section 2.6.2 and 2.6.3, those sections are referred to for any questions regarding the theory behind it. In this section the concepts of Embeddings, Positional Encoding, Feed Forward, Add & Norm and will be addressed.

Embeddings, often denoted as d_model:

Embeddings are numerical representations of words that can be used as input into a transformer or a text based neural network architecture, as words in themselves do not have any quantitative measurements to them. Typical values of choice are in the range of 256 - 768 but can definitely be higher as it depends on the complexity of the model. As each word can be interpreted in many ways, these are essentially what the embeddings represent. The d_model of a transformer network is one of the largest contributing factors to the amount of parameters in the model, as its value is multiplied with the size of the vocabulary of the model, meaning high value of d_model and vocabulary size quickly increases the computational costs of the model, sometimes without providing a drastic increase in value.

Positional Encoding:

Positional encoding is used as a mechanism for the transformer model to understand the value of distance between embeddings. Whilst attention is useful for the interpretation of how words can affect each other, and the connection between them, it does not value the relation of position between words. Thus positional encoding is introduced as an addition to the encodings to value the distance relation between the tokens (Phillips 2023).

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Figure 7: Formula for Positional Encoding (Tensorflow 2024)

Feed Forward:

The feed forward process is very much akin to the ones mentioned in 2.2-2.3 sections, it functions as a feed forward layer that enables the model to learn from the results by initially expanding the output of the multi headed attention layer to a higher dimension, enabling learning from a higher set of relations between the variables. After initial scale up, the connections from the first layer in the feed forward network are input into a ReLu activation function, and then scaled back to its original state. The main purpose, as mentioned, is to enable the model to learn from more complex features of the multi headed attention layer.

Add & Norm

The Add & Norm step applies residual connections to blend each layer's input with its output for continuity, followed by layer normalization to stabilize and accelerate learning by ensuring consistent scale across the network's depth. It is a way of adding the results of the new discoveries to the earlier data, whilst maintaining the values of the original input without losing any critical information. This is the reason why the arrows in the architecture go both around a layer in the network, but also straight into it. (Vaswani et al. 2017)

### 2.7.2 The Encoder Block

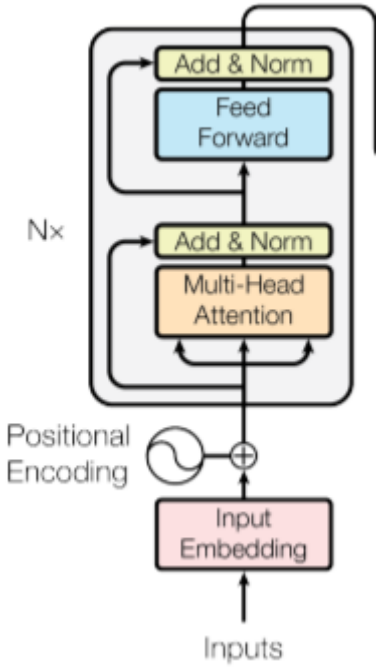


Figure 8: Encoder block of a transformer network (Vaswani et al. 2017)

The main purpose of the encoder block is to interpret the meaning of the input with regards to the embeddings, attention and positional encoding. One could think of its purpose as “how do I understand what this actually is and what it is saying?”. To relate to how people might see a text they read in a scholarly paper it might be that there is a narrative flow guiding the reader and displaying problems, situations, calculations and solutions. This together creates the narrative of the input; the chain of sentences builds a story from the data presented.

This is in essence what the encoder tries to capture. What is important in the input text and why? Why is it structured like it is? What are the most important words that others adhere to? (attention) How long into the text and backwards does these important words affect the story that's being told? (positional encoding) and what are the relations between the words and characters? How is it being told on the character by character and word by word level? (embeddings). The encoder tries to analyze the input, to provide the decoder with where to look for information that may affect the decoder's output, as the decoder's final output is dependent on the encoder. (Vaswani et al. 2017)

### 2.7.3 The Decoder Block

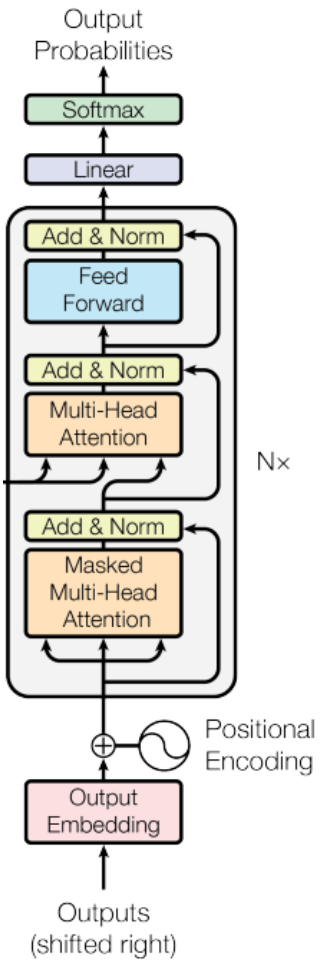


Figure 9: The Decoder block (Vaswani et al. 2017)

The decoder block is a bit more complex to understand, primarily due to the reason for the two multiheaded attention layers and the input it receives from the encoder block. If the encoder block can be seen as trying to understand what the input data is, the decoder block can be understood more as "what is important to focus on in the input data and why? How does the information about the input affect how we view the target data? What relation can be found between the two?". When visualizing this, interpret the input as the abstract to be summarized, and the target as the title which is the end goal of the model's output. A detailed thought process be found below:

1. Similar to the encoder we generate the embeddings and assign them positional encoding values to help the model keep track of the position of tokens in the target
2. The first attention layer picks up on the relation between the words in the target sequence, identifying patterns and the most important words to relate to that dictates the flow of abstract
3. In the second attention layer we inject the residual connections from the encoder's output into a new multi-headed cross attention layer together with the

information from the first multi-headed attention layer. This allows the second attention layer to combine the two different information sources to get a wider concept of how input and target relate to each other, and how the most important elements of the target relate to the input to form the entire data point. As in our case, how the body of text affects the generated abstract.

4. The feedforward layer and add&norm then expands the results of the attention layer to learn from higher dimensions of understanding from the attention layer, allowing the model to capture even more complex relations. Afterwards the results are generated and the loss is calculated.

Whilst above details the thought process of how the decoder works in a narrative flow, it is used a bit differently in training in contrast to validation of the data. Whilst the above description best suits the validation aspect, there is not a large difference between the validation and training. The decoder is the part of the network the predicts the next likely sequential output, as for example if the previous words were:

"As for"
The decoder might predict there is a very high probability the next word is "example"

So when we train the model to predict this next sequence we give the model "As for" and calculate the loss towards the correct prediction which is "As for example". This penalizes the model for generating the wrong token and makes the next token work as a smaller target within the model for next token prediction. This is referred to as teacher forcing and is commonly applied within the area of transformer decoder networks (Vaswani et al. 2017).

### 2.7.4 The full Transformer Network in “Attention is all you need”

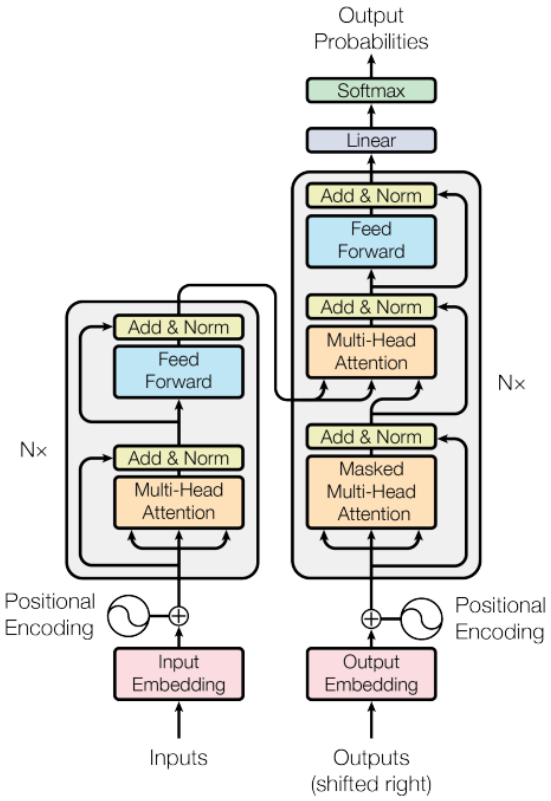


Figure 10: The full transformer architecture (Vaswani et al. 2017)

## 2.8 Variations of Artificially Generated Text Summaries

Artificially generated summaries in the context of this thesis refers to summaries generated and not written by a human. In this field of study there have been two distinct versions of text summarization, namely that of extractive text summarization and abstractive text summarization. How the concepts differ is that in the first case the sentences which the model deems most relevant are picked out and together form the summary, in the abstractive method the summary is generated from scratch by the model (Khatri, Singh, and Parikh 2018).

The extractive method of summarization might thus be regarded as less intelligent as it consists of outtakes of sentences from the original text. This can make the summary clunky and will add little in terms of variation to the reader. What the extractive method makes up for in terms of lower quality it does own up to in terms of computational cost and efficiency. The sole purpose of an extractive model is to learn what sentences are of the highest value in a text and then generate the summary, this is much easier than the demands on the abstractive method which must learn to form sentences, understand words, and identify key patterns in the original text (Khatri, Singh, and Parikh 2018). An extractive method can be of value if the intent is to output a rough understanding of

the narrative, but for higher quality the choice of model should reasonably be abstractive.

The abstractive method of summarization generates new sentences and words to create the text summary. As stated above this operation is vastly more complex and costly than the extractive but with a well-functioning network the summary output could be of higher quality, easier to read and comprehend as well as provide something akin to that of a summary written by a person. What this thesis will focus on is an abstractive method of text summarization using a encoder and decoder transformer network

## 2.9 Related Work

The primary source of inspiration for this study is from the critically acclaimed and groundbreaking paper “Attention is all you need” which is the first public record of the invention of the transformer model (Nallapati, Zhai, and Zhou 2017). In the Attention is all you need paper Nallapati et al. build a transformer model for the purpose of language translation, which has similar challenges to that of text summarization but with some critical points of difference.

Both the network presented in this study, and the one used by Nallapati et al. utilize a transformer encoder - decoder network, but the key difference is that this study’s goal is summarization instead of language translation. The fundamental model and scheme is the same as the one in Nallapati et al.s study, albeit with different parameters and reasoning behind the changes in mind. For that reason, it's highly encouraged to revisit Figure 10 from time to time to synchronize the reading with the relevant layers of the transformer.

Another relevant study, albeit not for the same purpose as building the transformer network, is the one by Brown et .al in the article “Language models are Few-Shot Learners” which presents parameters for some of the GPT models. These choices of parameters are of interest for drawing comparisons in the models training and evaluation process, as these are the most direct parameters that affect the models ability to comprehend complex high level patterns, and train efficiently. Something worth noting already is that the models presented have capabilities and parameters greatly beyond the scope of the computational costs at use for this study, which is something that will be discussed in later sections. The table of parameters can be seen in Table 1.

| Model Name | $n_{\text{params}}$ | $n_{\text{layers}}$ | $d_{\text{model}}$ | $n_{\text{heads}}$ | $d_{\text{head}}$ | Batch Size | Learning Rate |
|---|---|---|---|---|---|---|---|
| GPT-3 Small | 125M | 12 | 768 | 12 | 64 | 0.5M | $6.0 \times 10^{-4}$ |
| GPT-3 Medium | 350M | 24 | 1024 | 16 | 64 | 0.5M | $3.0 \times 10^{-4}$ |
| GPT-3 Large | 760M | 24 | 1536 | 16 | 96 | 0.5M | $2.5 \times 10^{-4}$ |
| GPT-3 XL | 1.3B | 24 | 2048 | 24 | 128 | 1M | $2.0 \times 10^{-4}$ |
| GPT-3 2.7B | 2.7B | 32 | 2560 | 32 | 80 | 1M | $1.6 \times 10^{-4}$ |
| GPT-3 6.7B | 6.7B | 32 | 4096 | 32 | 128 | 2M | $1.2 \times 10^{-4}$ |
| GPT-3 13B | 13.0B | 40 | 5140 | 40 | 128 | 2M | $1.0 \times 10^{-4}$ |
| GPT-3 175B or "GPT-3" | 175.0B | 96 | 12288 | 96 | 128 | 3.2M | $0.6 \times 10^{-4}$ |

Table 1: Parameters in different GPT-3 models

# 3. Method

In this section the methodology and implementation of the summarizer algorithm is detailed, and a thorough breakdown is conducted. This section's purpose is thus to explain the thought process behind the final model and which decisions were taken into consideration upon reaching the models final state.

## 3.1 Why Abstract to Title summarization

Albeit it might seem a bit unimpressive at first thought, the scholarly papers found in the CORD-19 dataset contains vast amounts of data with very difficult and varying text. Plenty of academic and mathematical expressions, different fields of study and varying language. With that in mind, it's too large of a task to perform summarization over the entire data as the computational expense is enormous for these types of calculations over thousands of words as input at once. With this in mind, the course of the study is slightly adjusted towards the relations between the abstracts and the title, which nevertheless is an important stepping stone towards generalizing the model for text summarization of the entire papers.

The model trained from this procedure can then be continually improved upon with fine tuning and hierarchical summarization to summarize sections of each paper's text body which are then summarized into an abstract. Intuitively, the title can still be said to in most cases represent a summary of the abstract, which in turns represents the text content of a paper. With this in mind, and with the goal to still create a somewhat functional model within the constraints, the approach was changed from text body summarization to abstract summarization for generating short and cohesive titles representing what a paper contains.

## 3.2 From Extractive to Abstractive summarization

Initially at the beginning of the thesis formulation the idea was to approach an extractive text summarization model, due to limitations such as scope of the study, technological limitations and lack of scientific research on the subject at the time of the studys conception in 2020. However, with the eminent success of ChatGPT the scientific literature on the subject grew from 2020 to 2024. Enabling an easier point of initial learning of the fundamentals of the technology, and solidifying the architectural approach for the thesis problem definition.

With the above reasoning and the factor of abstractive text summarization being a more intricate and challenging task, abstractive summarization was the finalized idea. Also, in the era of ChatGPT and transformer networks the idea of extractive summarization is no longer as alluring, as its result may be far worse than what a transformer network can produce with its abstractive generation, and is less technically intriguing.

## 3.3 High level explanation of the Model and the Data

The data that is used for the transformer summarizer model is taken from the CORD-19 dataset. The dataset contains several more parameters than necessary for each paper, therefore each paper is cleaned of unnecessary contents and purely the abstract, paper id and title of the papers are left, saving the raw data to a json file for preprocessing.

The data is then moved through a preprocessing pipeline to prepare it adequately for the transformer model. In this step, several methods are used to clean the text of unnecessary tokens and validate that the data is somewhat uniform to enable the best premises for the model to learn from. After preprocessing the data is moved into the tokenizer which will be referred to going forward as CoTo, which converts each word into a numerical representation in the vocabulary, with special treatment for tokens that are found less than 5 times in the dataset. Each word that is found less than 5 times is mapped towards a special token called <UNK> which represents words that are very rare, and therefore can be assumed to have little value in the grander scheme of training. Also, start of sequence token (<s>), end of sequence token (</s>) and padding tokens (<PAD>) are added to the data. The data processing and tokenizer is further developed in Section 4.

After tokenization, the transformer network utilizes the encoder part of the network to encode the importance and aspects of the sections in the abstract. The output of the encoder will then together with the target combine into the second multiheaded attention layer in the decoder network, enabling a feature encapsulation of the relations between the abstract and a paper's title, with the output being a set of predictions based on the probabilities and relations established through the training procedure.

## 3.4 Why Transformer model instead of RNN?

After deciding upon the abstractive approach, I had come to learn that the previously dominant and most popular method of approach was RNN model using attention. As discussed in section 2.4, the RNN approach does involve quite a few flaws the transformers don’t have. Most likely a reason for the dominance of RNN: s in the current scientific literature is that the concept of a transformer is quite new and was originally as a concept published in June of 2017. (Vaswani et al. 2017)

With the upsides of the transformer such as parallel processing and better suitability towards NLP tasks the choice for the transformer seemed like the most suitable approach towards an abstractive text summarization model, additionally the transformer networks of today are still being heavily developed on a wide variety of tasks in the NLP field, making scientific contribution on the subject all the more relevant, and by today's standards outclasses the RNN:s in many tasks as, discussed in the theoretical section.

## 3.5 Hardware, Design tools and Development environment

The project was developed in Visual Studio Code using Python 3.12.1. The transformer model was built primarily using the PyTorch library for setting up the architecture. Computations were primarily done on the GPU to increase the speed of the calculations using PyTorchs CUDA library, specific hardware is found below:

Processor: AMD Ryzen 7 5800 Processor

Memory: 512 GB SSD, 16 GB RAM

Graphic Card: Nvidia Geforce RTX 3060

During heavier training procedures hardware from Kaggle's cloud service for data development was used, enabling access to a GPU P100 which is a powerful GPU that highly advanced the computational power accessible, however only accessible 30 hours a week, somewhat limiting the scope of its usage.

For version control Git was used to a private repo where the code contents remain private indefinitely, upon request snippets may be reviewed at leisure of interest.

## 3.6 Dataloader, Optimizer and Loss Function

The project utilizes PyTorchs data loader library to maximize the efficiency, batching and loading of the data for input into the transformer. The project utilizes the commonly accepted Adam Optimizer also found in the PyTorch library, with a weight decay of 0.01, which implements L2-Regularization to prevent overfitting onto specific large weights that might dominate a sequence, which might definitely become an issue in text generation tasks. The loss function that is used is cross entropy loss, as the transformer outputs logits is an effective tool for estimating the correctness of the models predictions towards the ground truth.

## 3.7 Model parameters

| Parameters | Values |
|---|---|
| batch_size | 16 |
| input_block_size (abstract length) | 148 |
| target_block_size (title) | 28 |
| vocab_size | 18211 (determined by tokenizer) |
| d_model | 516 |

| | |
|---|---|
| num_heads | 12 |
| num_layers | 4 |
| dropout | 0.1 |
| Initial learning rate | 0.0002 |
| max_len_positional_encoding | 256 |

Table 2: Hyperparameters

## 3.8 Regarding choice of parameters

In this section some of the more impactful choices of parameters are discussed for the sake of motivating the structure, with regards to architecture of the transformer.

### 3.8.1 Learning rate

The transformer model uses a learning rate scheduler from the hugging face transformer library, namely; get_linear_schedule_with _warmup, as the function is denoted. The equation can be found in the python library and was retrieved from the source code 2024-04-02.

The learning rate is dynamically adjusted during the training process to prevent the model from too quickly identifying specific patterns, but also to prevent overfitting of the model after long training sessions. It uses the following formula:

1. Denotations

Learning rate = lr

Timestep = t

Warmup_steps = 0.1×total_steps

Total_steps = length(dataloader) × num_epochs

Initial_lr = 0.0002

2. During warm up

lr = (initial_lr × t) / warmup_steps

3. After warm up (t > warmup_steps):

lr = initial_lr × (1 - (t - warmup_steps) / (total_steps - warmup_steps))

### 3.8.2 Hyperparameters: d_model, num_heads and num_layers

d_model, num_heads and num_layers are the three most critical hyperparameters for tuning the model's ability to learn advanced patterns in the data. Num_heads allows for different heads of attention to learn from the data as can be seen in the figure 6 in section 2.6.2, allowing for the high dimension of complexity from the d_model for each token to be more critically examined. This allows the attention to find patterns which might otherwise be too complicated to identify, and for this splitting of heads to work, the value of d_model needs to be divisible by the number of heads. The final parameter, num_layers, sets up the encoder and decoder architectures num_layers amount of times, allowing for aggregation from the findings of each layer at the end of each passthrough into the encoder and decoder. Since the encoder and decoder each apply randomized dropouts to prevent overfitting, the aggregation allows for a specific sample of data to be more thoroughly examined as each layer performs its own computations before the results are aggregated. Num_layers can be found denoted as Nx in figure 10 in section 2.7.4.

### 3.8.3 Total amount of parameters

In a study done on the GPT-3 models from 2020 the model with the most parameters was trained with upwards of 175 Billion parameters, which is far beyond the scope of this study's capabilities (Brown, 2020). Below are calculations of parameters for the study's model presented.

1. Multi-Head Self-Attention (MHSA) - Encoder and Decoder

Query, Key, and Value Matrices (Q, K, V):

- Each head requires a separate projection matrix for Q, K, and V.
- For each head:

  $3\times(d_\text{model}\times d_\text{head})$

- Total parameters for all heads (including final projection layer):

  $3\times(d_\text{model}\times d_\text{head})\times\text{num_heads}+(d_\text{model}\times d_\text{model})$

- Calculation:

  $3\times(516\times43)\times12+(516\times516)=1\ 065\ 024$

2. Feed-Forward Network (FFN) - Encoder and Decoder

First Layer:

- Expands the input features to four times the model's dimension:

$d_{model} \times (4 \times d_{model})$

- Calculation:

  $516 \times (4 \times 516) = 1{,}065{,}024$

Second Layer:

- Projects back to the original model dimension:

  $(4 \times d_{model}) \times d_{model}$

- Calculation:

  $(4 \times 516) \times 516 = 1{,}065{,}024$

Total FFN Parameters:

- $d_{model} \times (4 \times d_{model}) + (4 \times d_{model}) \times d_{model} = 2{,}130{,}048$

3. Encoder and Decoder Layers

Encoder:

- Each encoder layer contains both MHSA and FFN.
- Total per Encoder Layer:

  attention_params+ffn_params

- Calculation:

  $1{,}065{,}024 + 2{,}130{,}048 = 3{,}195{,}072$

Decoder:

- Each decoder layer contains masked self-attention, cross-attention, and FFN.
- Total per Decoder Layer:

  self_attention_params+cross_attention_params+ffn_params

- Calculation:

  $1{,}065{,}024 + 1{,}065{,}024 + 2{,}130{,}048 = 4{,}260{,}096$

4. Final Sum of All Components

- Embedding Layer:

- Input Embedding and Target Embedding:

  $2\times(\text{vocab_size}\times d\text{model})$

- Calculation:

  $2\times(18{,}211\times516)=18{,}793{,}752$

- Overall Total:

  embedding_params_total+encoder_total_params+decoder_total_params

- Calculation:

  $18{,}793{,}752+12{,}780{,}288+17{,}040{,}384=48{,}614{,}424$

# 4. Data

In this section the data used for training and evaluating the model is presented along with an example outtake of how the preprocessing was done to prepare the raw data for usage in the model.

## 4.1 Dataset: COVID-19 Open Research Dataset (CORD-19)

The CORD-19 dataset is a collection of scholarly papers created by the White House and a coalition of leading research groups to provide the AI and ML community with resources to aid in the ongoing fight against the coronavirus pandemic. The dataset consists of around 401 000 papers on the subject as well as embeddings, pdf/pmc format papers, data points in terms of population, countries, risk factors and more. The specific subject and scholarly discipline in each study is highly varied and may be labrapports, published papers, or general discussions, within for example chemistry, biology, mathematics and AI.

## 4.2 Data Preparation

In this section the details for how the data was preprocessed and implemented are presented. A split of 5% validation and 95% training was chosen, as the number of useful samples postprocessing were around the low 20-thousands.

### 4.2.1 Data preprocessing and the parameters

Seeing as how the original dataset contains a lot of redundant data for this study such as tables, citation, references and more the data first needs to be processed to remove unnecessary parameters which might cause extensive computational cost and increase memory occupation unnecessarily.

For the outline of the study and the transformer summarizer only three parameters of each study are deemed valuable: paper_id, title and abstract. Paper_id is optional but serves a purpose for debugging and identifying a specific paper, which also contributes towards easy mapping into a json file with the id acting as the key.

All the papers utilized for the training and evaluation of the model have been vetted to contain defined values for the three selected parameters.. After extracting the samples post processing methods were implemented to clean all the papers from content which might disrupt the training process of the transformer, but without hurting the robustness of the model. The following are the major pre processing steps that were implemented for each paper, but more minor adjustments were also made after finding specific faulty patterns in the data:

**Language:**

Each paper was language checked with the python library langdetect to only contain English - including several different languages in the samples will increase the robustness of the model, but require a lot more data to support and is not essential for the models performance. The study's aim is to purely determine that capability of summarizing English texts, thus any studies found to be non-english were removed. Where the line is drawn between English and non-English is difficult to pinpoint as things such as names and cities might be in different languages, but only a handful of papers were removed using this method, therefore it was deemed unharmable procedure.

**Characters and words:**

The CoTo tokenizer uses wordbased tokenization, therefore a minor procedure of eliminating rare words was implemented, classifying all words appearing less than 5 times in the texts in total as <UNK>. This is a common procedure for teaching the model to redirect its attention towards the more central wordings, both reducing the size of vocabulary, but also drastically improving the models ability to be precise in its generation. The vocabulary in the end reached a size of about 19000 unique words, which appeared more than 5 times each. In this vocabulary the special tokens known as start of sequence <s>, end of sequence </s> and <PAD> was also included. The line between where to identify words as <UNK> tokens not is somewhat difficult, especially when not a lot of data is involved. For example the model could be prone to generate the unknown token too much, or too many words isolated to the token.

**Sources:**

In many of the papers they contain references to sources which do not provide any interpretation value for the transformer model and may reduce the effectiveness of the training algorithm, for that sake some of the most common source reference models very removed, namely:

[x] - digit source citation

[x] - [y] - multiple source citations

[x, y, z … ] - multiple source citations, different format

A lot of other regex processing was also implemented, too many to include here but upon request could be submitted.

### 4.2.2 Showcase of exemplary data before tokenization

The end product after extracting only the necessary data can be found in Figure 8 containing the paper_id, abstract and title. All the around 20000 papers that were used were processed in the same manner to be prepared for input into the tokenizer.


```
"0fa8fc2dcbcfa3c1ebae2e547a27e22068e12015": {
    "title": "the horrors of covid-19 and the recent macroeconomy in ethiopia",
    "abstract": "this review paper is emphasized on covid-19 and macro-economy like gdp, unemployment, inflation and macro-econ
},
```


Figure 11: Example of processed data

### 4.2.3 Data outliers

Some of the data contains misleading characters, language and structure. Whilst this is an inevitable issue of large amounts of data, there is also a benefit in having some problematic data as it increases the robustness of the model. Even if possible to remove papers containing troublesome or unknown characters, it would make the model much less effective at handling unknown data. Furthermore, in terms of scientific literature akin to the studies on the Corona pandemic, a lot of unique symbols tend to appear for mathematical and variable purposes. There are also bound to be some samples in the data that are entirely wrongly processed or created (for example when pre-analyzing data charts of the distribution of titles and abstract relations, some samples of titles were around 1200 words, which in all likelihood is impossible for a realistic title). For this reason the title lengths were limited to between 4 to 16 words, whilst the abstract were limited in the range from 48 to 156 words. According to the scatter plot in Figure 12, it was one of the most heavily clustered areas of somewhat uniform data. The reasoning behind this limitation of the size is that they might intuitively make sense for somewhat condensed papers, whilst there exists a somewhat reasonable relation between the sizes of the two. For example, going from an abstract of 1000 words into a title of 6 words, might pose a bit of a challenge for the model if most of the other samples are in the range of 48 to 156. Somewhere, a reasonable limit is best to take, and it fell upon this for the reasons previously mentioned in addition to reducing the computational costs.

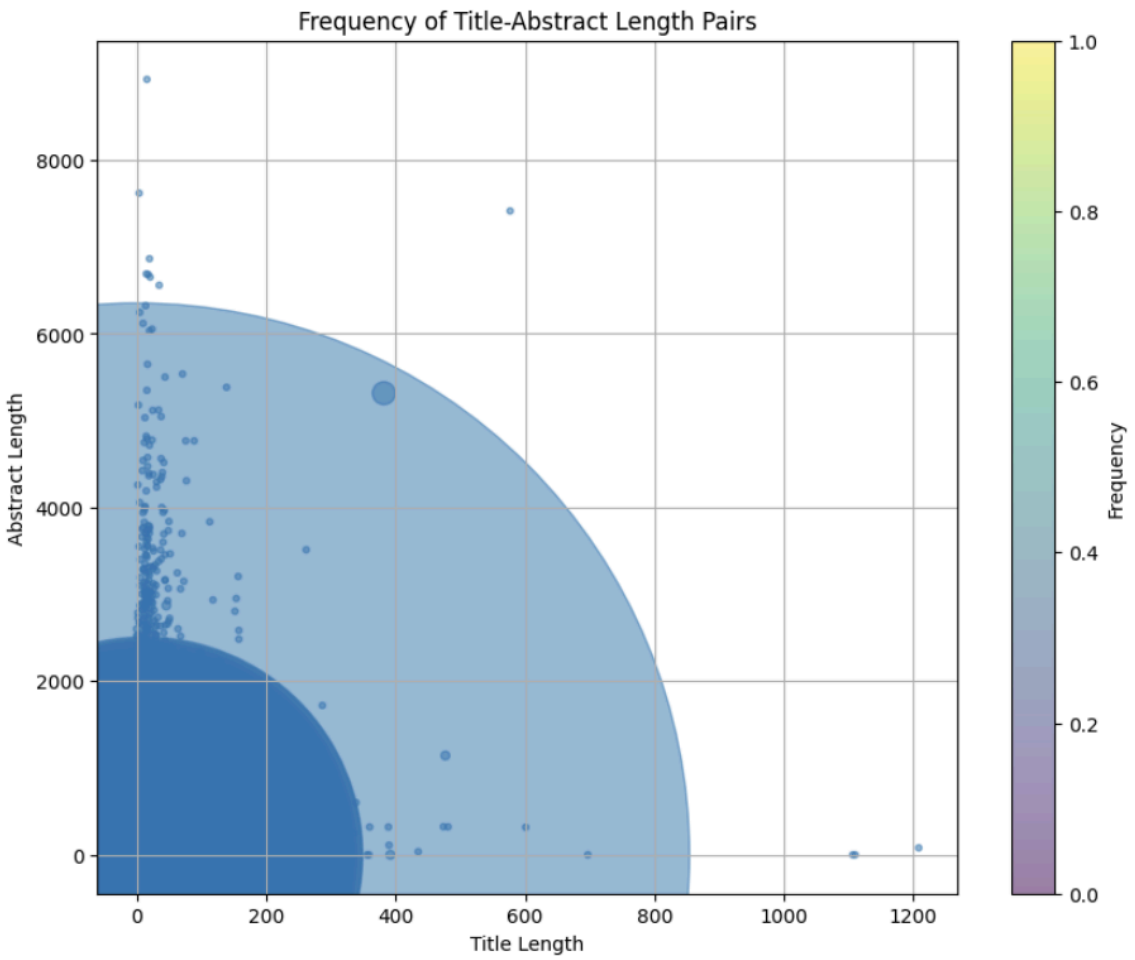


Figure 12: Distribution of title and abstract lengths in the CORD-19 dataset

### 4.2.4 Tokenization of the data

After the processing and cleaning of the data, and before submitting the information to the transformer model, tokenization was done to correctly transcribe the word representations into usable data by the model. The CoTo tokenizer was used for this purpose, and is a simple tokenizer which converts words into tokens according to the regex rule found in figure 13. For example . , ! ? are treated a special words, together with <s>, </s>, <PAD> and <UNK>

```
def encode(self, text):
    """Tokenize a text into tokens considering punctuation."""
    tokens = re.findall(r"\w+|[^\w\s]", text, re.UNICODE)
    return tokens
```

Figure 13: CoTo word identifier

After having identified and split up all the words into tokens, each token is designated its numerical representation mapped through a dictionary, netting a finalized vocabulary size of around 18000 tokens.

```
sentence, _ = tokenizer.tokenize("then there how are you", add_special_tokens=True, max_length=10)
print(text)
print(f"The tokenized version: {sentence}")
print("And the decoded tokenized version: ", tokenizer.decode(sentence))
```

Figure 14: Tokenization code

```
then there how are you
The tokenized version: [0, 464, 133, 101, 26, 3821, 2, 1, 1, 1]
And the decoded tokenized version:  then there how are you
```

Figure 15: Showcase of CoTo tokenization, result of Figure 13 code

In Figure 15, a showcase of how CoTo works is presented. Worth noting is that each different tokenizer tokenizes text and data differently according to the developers best suited needs. As mentioned earlier in 2.5 there are many much more sophisticated, trained and developed tokenizers that are optimized for specific transformer models needs, but as was outlined in 1.2 this is out of the scope of this study. 0 marks the beginning of a sequence, 2 the end of a sequence and the 1:s are padding, which are important for masking when inputting into the transformer. This essentially tells the transformer to ignore these positions when doing calculations as they are just filler values to reach a target length.

### 4.2.5 Batching

Batching is a crucial process to improve the efficiency of the training algorithm in many deep learning models, and it holds true for the transformer based models as well. Batching is a process of allowing parallel processing in algorithms to break down large amounts of data into chunks that can be processed in parallel. This allows the model to train on several batches at the same time, improving efficiency but at the cost of computation power. It however also comes with another benefit of increased convergence as can be seen in Figure 16.

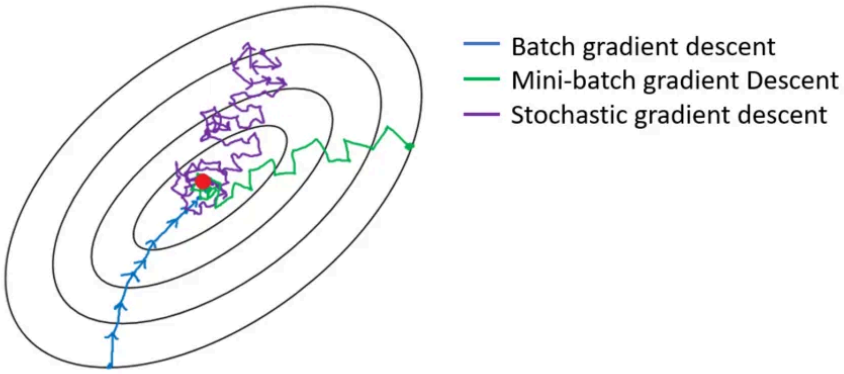


Figure 16: Examples of batch processing and effect on convergence (Sweta 2020)

In Figure 16 the red dot symbolizes the point of convergence, and whilst evident that the stochastic gradient descent approach of handling one sample at a time provides a lot of noise, it's not feasible for utilizing parallel computation, which gives it two huge downsides. The batch gradient descent is also troublesome, as it might put too much strain on the computational cost of calculating such large scale batches at once. A common middleground for selection which is also utilized in this study is the mini-batch gradient descent where we use a sample size of about 16 samples. The benefit of this is that before updating the parameters of the model, we process the teachings of all the samples in unison, preventing bad samples from overfitting the model in the wrong direction as the results are averaged across the batches, whilst also not putting an enormous strain on the computational front. In this case, a sample in the batch represents the title and abstract of one paper, meaning that a batch of 16 samples consists of 16 papers, each with their unique abstract and title. There is good reason to opt for a larger size here, around 32 to 64, but the computational restraints were out of the possibilities of the study.

# 5. Results

This section will showcase examples of generated summaries together with the real titles for the papers. Some values from the training process will also be shown such as the loss metric for discussion in Section 6, which is important for evaluating the progress of the network and how it theoretically could develop with an increase in training time and complexity of the transformer architecture.

In general, the model ran into problems of diversifying its content towards the data, which might be indicative of the model not being exposed to enough data. For example, it has definitely picked up on patterns such as the length of summaries, how to start and end a sequence, but the content it generates does not properly reflect the content it's fed and the actual target.

## 5.1 Qualitative results: Samples

To get a representation of the performance of the transformer, qualitative results can be extracted in the form of generated titles. From a mathematical perspective, there is currently no precise performance measurement used to analyze the qualitative results and their correlation to the actual titles. This was selectively chosen to be ignored as the model not yet reached capabilities where it generates unique titles that are of any relevance to compare to currently, but measurement such as F-score, precision or recall could be used to measure the similarities in a qualitative manner, but is not done for these specific samples, as they are generic in nature.

### 5.1.1 Three AI Generated summaries and target Abstracts

```
Paper ID: 0c56b7efe71241074735aadbdc487e472377cac7
Generated Title: the covid-19 pandemic: a case report and covid-19 pandemic
Actual Title for paper:
how big data and artificial intelligence can help better manage the covid - 19 pandemic
```

Figure 17: Generated summary 1

```
Paper ID: 6bf54dacb8c1deb04c6f41363dd6b4b1e48a8945
Generated Title: the covid-19 pandemic: a case report of the covid-19 pandemic
Actual Title for paper:
physics of fluids article scitation. org/journal/phf jet fans in the underground car
parking areas and virus transmission
```

Figure 18: Generated summary 2

```
Paper ID: 0cf53d6709595f2362eec8f4a169d305f2f3f513
Generated Title: the covid-19 pandemic: a case report of the covid-19 pandemic
Actual Title for paper:
sequencing of coronavirus ibv genomic rna: a <UNK> open reading frame encoded by mrna b
```

Figure 19: Generated summary 3

Worth noting, although perhaps a bit hard to see, is that the summary in Figure 17 is a bit different from the others, containing the word “and” for example.

## 5.2 Quantitative Results: Loss calculations

Quantitatively, the results can be examined in terms of batches and epochs. This allows for a more mathematical approach that can be generalized and compared to, in contrast to the qualitative samples in Section 5.1. The loss calculations are the accumulation of errors in terms of the logits generated by the model, this can be explained with that the model calculates the error distance between the actual generated next words, and contrasts it to the models probability of each token. This means that the loss shown in Section 5.2.1, is a quantitative accumulation of the errors generated during each epoch, meaning a lower loss will indicate higher performance, leading to higher qualities of generated summaries.

### 5.2.1 Loss calculation: Training set

```
Epoch 3, Batch 1000/1323, Loss: 5.5341
Epoch 3, Batch 1050/1323, Loss: 5.4233
Epoch 3, Batch 1100/1323, Loss: 5.6142
Epoch 3, Batch 1150/1323, Loss: 4.9881
Epoch 3, Batch 1200/1323, Loss: 5.2512
```

Figure 20: Batch Loss calculation Epoch 3

```
Epoch 5, Batch 50/1323, Loss: 5.2007
Epoch 5, Batch 100/1323, Loss: 4.9578
Epoch 5, Batch 150/1323, Loss: 4.6434
Epoch 5, Batch 200/1323, Loss: 6.0276
Epoch 5, Batch 250/1323, Loss: 4.7557
```

Figure 21: Batch Loss calculation Epoch 5

```
Epoch 6, Batch 50/1323, Loss: 4.9147
Epoch 6, Batch 100/1323, Loss: 5.4227
Epoch 6, Batch 150/1323, Loss: 5.4673
Epoch 6, Batch 200/1323, Loss: 5.5212
Epoch 6, Batch 250/1323, Loss: 5.6409
```

Figure 21: Batch Loss calculation Epoch 6

### 5.2.2 Loss calculation: Validation set

```
Epoch 3, Batch 800: Validation Loss: 7.6481
```

Figure 21: Validation loss Epoch 3, at batch 1200

```
Epoch 4, Batch 400: Validation Loss: 7.4092
```

Figure 22: Validation loss Epoch 4, at batch 1200

```
Epoch 5, Batch 400: Validation Loss: 7.2608
```

Figure 23: Validation loss Epoch 5, at batch 1200

# 6. Discussion

In this section, I will critique the models performance with regards to the loss development during training and the outputs in the generated summary. Furthermore, I will also perform a technical analysis of why the results are what they are, what they indicate and both review and reflect of the models architectural design in relation to the task assignment and the problem definition the study was originally created for tackling.

## 6.1 Performance comparison to actual titles

The model definitely does not perform at the rate to what would be desirable. It opts to in most cases generate a generic title of a paper, which definitely indicates that it has learnt something from the data (evidently the papers are about covid!) but it has not been able to pick up on the specific patterns of each paper and what makes each paper stick out from the rest. There are a couple of reasons for this, the first and most prominent one is that most likely the model has trouble finding the actual patterns of what data is the most important for translating the text contents into a title, one might suggest that this issue lies in the fact that the small dataset of 20 100 papers, especially with the high variety in content, may be a leading issue towards the generalization of generated titles.What further attributes this claim is that the model appears to over time be performing really well on the training data and improving for each epoch, reaching low losses for batches about 4.5 as can be seen in Section 5.2 (there were lower losses as well, but around 4.5 seemed to generally appear), meaning that patterns of sequences it has seen previously it has learned from and understood, something that is then reaffirmed in other samples which leads to the occasionally low loss rates.

In contrast to the low training loss, the model sometimes also spikes a bit towards 5-6 cross entropy loss for a batch, this might be a bad luck sample of many invalid data samples appearing in the same batch, but it is also most likely indicative of that the there are some patterns in the data which are not represented at all in other papers, as will be showcased in the following section 6.2.

What can be deferred though, is that there are clear signs of learning in the model, for example it has picked up on the fact that most papers if not all are assumed to be on the topic of the corona pandemic and are studies that investigate said pandemic. The model also shows the ability to properly generate end of sequence tokens resulting in titles of about 6-10 words, which one might find reasonable and can be seen in the figures in section 5.1.

The following sections will examine this reasoning more thoroughly, looking at both the data, the architecture and how the approach towards the data could be adjusted to accommodate to the challenges that were encountered.

## 6.2 Critical Discussion of the data

Whilst the results of the transformer might not seem very impressive, there is a good theoretical reasoning for this issue which mainly relates to the CORD-19 dataset and training a transformer from scratch on that particular data. As shown in the scatterplot in Figure 12 the distribution of abstract and title lengths are quite varied, making it problematic to extract cohesive data although there are 401 000 samples in the dataset. With the choices of title lengths in the range 4 to 16 and abstract lengths in the range 48 to 128 only about 21 000 usable examples were able to be used in training the transformer, which is considered quite a small sample of usable data There might be some combination of ranges that would yield a higher set of data whilst still being somewhat similar, but as shown in Figure 11, the distribution is quite varied.

With the limitation of amount of data being a factor, another one was that of each data point in itself. As earlier proclaimed, there is a lot of varied content, formulations and scientific disciplines spread throughout the papers, meaning that there is a wide variety in the types of data presented. Whilst this would not be as much of an issue if there was more readily accessible data, it definitely presents an issue for the small dataset that was used. There is also a lot of data that can be quickly identified just by scrolling to appear somewhat misleading even from the perspective of a human reviewer, an example of this can be found in Figure 24.


```
"43e614437f8f470e8e716efdab22875e2d1aaec8": {
    "title": "0123456789) 1 3 the journal of obstetrics and gynecology of india",
    "abstract": "the outbreak of covid-19 has become a globally concerning pandemic hav
},
```


Figure 24: Showcase of bad data

The title does not really make any sense and might be the error of when the data was downloaded or extracted. Many examples akin to this appears in the CORD-19 dataset, and whilst actually many of these errors are found in different samples, the example in Figure 24 of 0123456789) actually occurs in 91 other papers, thus to further clean the data, patterns like these would need to be more thoroughly identified in the dataset.

Furthermore, some definitely difficult to comprehend examples that are outliers in terms of understandability are readily found in the data as showcased in Figure 25 and Figure 26.


```
"0be429c6a1cf98ad469b6c369e450cfd93b5b8cf": {
    "title": "an m/ph/1 queue with workload-dependent processing speed and vacations",
    "abstract": "motivated by the trade-off issue between delay performance and energy
},
```


Figure 25: Showcase of ambiguous data

Although I, the author, is lacking a bit in the biological terminology, the sample in Figure 25 itself feels like an odd title, and if nothing else it certainly separates itself quite a lot from what one might think a regular title to be. It may be correct, but the importance for the model is; how many other samples like this one are actually in the data set?


```
"f5144a2ed86aef7190ef7d50973489f54acad2e4": {
    "title": "apni jung against ra patient empowerment: apni jung against rheumatoid arthritis for south asian population",
    "abstract": "covid-19 has affected many populations in the uk, and ethnic minority communities in particular. people from
},
```


Figure 26

Figure 26 is another case of highly specific data, the word apni and its use in the papers occurs only in this paper out of the 21 000 used, understandably the model might be confused when training on samples like these that are almost completely standalone and shares little both in terms of vocabulary, but also findings and the studies purpose. Outliers like these are good for the model to create a generalized perspective, but that is under the assumption that enough data is achieved to identify unknown patterns beforehand.

It does not, judging by the small sample of data and the variance in it, feel like a grand assumption to say that the prerequisites for properly training a model with about 60 million parameters under the circumstances would regardless of architectural choices of hyperparameters be troublesome.

## 6.3 Critical Discussion of the model

The most impactful hyperparameters utilized by the model, namely: num_head, d_model, vocab_size and n_layers were chosen with regards to the complexity of the task at hand and the difficulties of learning the language patterns. One could for example have reasoned that a lower number of heads and layers would be an advantage whilst increasing the batch_size to provide more stable gradient updates, but a counter argument is found in the examples of the data being quite complicated in itself with a wide variety of topics. It is hard to know though exactly what would be the best choices, it might be the case that the model is actually too advanced for properly learning to understand the data, seeing as how the size of the abstracts and titles are quite small.

In the end, they are referred to as hyperparameters for this very reason, it's difficult, next to impossible, to know exactly the best values to choose for the network, and it is specific to the data and task at hand and an iterative problem. The models were tuned during the training and varieties of layers, heads and d_model were experimented with, for example what happens if we increase d_model to 768, but reduce the num_heads and n_layers a bit? In the end, the architectural design presented in section 3.6.2 was chosen but definitely something that could be experimented with further to increase performance.

## 6.4 Ethical Aspects

Whilst the data processed from the CORD-19 dataset is open source for the purpose of community contribution, there is a different perspective in that the scientists who contributed their research to the dataset are also not credited properly for their work. To take a concrete example in relation to this study, the author serves no purpose to the model's training process or its performance, meaning that although the scientists' work are used for the model, they are never credited for the study's results or the model. This is a problematic aspect of all machine learning and big data models, and is something worth taking into consideration. With this in mind, I, as the author, would like to extend my gratitude to each and all the researchers contributing their work to the CORD-19 dataset and thank them for allowing their research to be open source and integrated into the CORD-19 dataset, as it has provided a fundamental aspect of being able to perform this study.

## 6.5 Possible improvements

As outlined in general, the largest factor would be to introduce more data and preprocess it further, removing for example redundant patterns and perhaps removing data where a certain word was only used in that paper. There is a lot of cleaning that can be done, and will have a huge impact on how the model learns. A way to avoid this sensitivity to outlier data would be to use a higher batch_size, but as the computational resources were already at their limit, that would mean downgrading in other areas.

A somewhat refined approach to the task of text summarization could also be taken by first approaching simpler language samples such as the CNN/Daily Mail dataset which is a classic dataset for the sole purpose of learning models to achieve text summarization. (Hugging Face, n.d.) When the transformer then has learnt these patterns which are quite a lot simpler and the idea of creating coherent language and generating summaries, it could start learning on the more difficult patterns found in the CORD-19 dataset. In that case, the 21 000 papers that were used in this study might actually prove sufficient for the model to have it generate better summaries. One could imagine applying transfer learning for this case, or training the model the same way but with a rather low learning rate such as to not lose the original strengths of training on the CNN/Daily Mail dataset.

The previously mentioned approach could also be of high value in the case of choosing to pursue the original task of summarizing the full body of text into an abstract. As has been mentioned earlier, the concept of hierarchical summarization could be of great value for this purpose, and having the model then pretrain on simpler data to learn the task could be of great value, as there won't be as readily of an access to purely correct data to train on for the long texts. There is also another alternative to this approach which is to use Retrieval Augmented Generation (RAG) to identify and pick out unique features that then are summarized to construct an abstract.

There are many ways to go about the continuation of the study, but they all consume quite a bit of time and computational power, which are all in short for the scope of this thesis.

# 7. Closing Remarks

## 7.1 Conclusions

Text summarization in itself is a very complex task of which a lot of day to day people struggle as well in performing, the functionality to adequately represent and summarize texts, even your own texts, is a difficult task. Details may be overlooked, concepts not thoroughly explained and the summary might not encapsulate the full value of the original text.

This perspective holds true to the transformer network case in this study as well. The loss value is positively improving with training, but the lack of somewhat uniform and learnable data prevents the model from properly conforming to registering patterns, and instead choosing to go for more generic outputs. As discussed in Section 6 there are a couple of major reasons why this might be the case. Summarization is a difficult task, and starting from no language knowledge at all and training to perform task summarization might be too difficult of a task for such a small sized network as the one utilized in the study and on such a varied and small data set.

Regardless of this, it's evident that the model is showing clear signs of improvement and the summaries have relations to the actual papers, but if the task of training a transformer for text summarization would be redone again with more computational resources and with the insights taken from the study, the approach would be revised primarily with regards to dataset and opt towards the more traditional datasets for summarization such as the CNN/Daily Mail dataset.

The study set out to examine the following three criteria in its problem definition in Section 1.1:

- How well does a Transformer model trained and built from scratch perform at the task of abstractive text summarization on the CORD-19 dataset?
- What teachings can be derived from building a transformer model from scratch?
- How can this study's research be built upon and what areas of possible improvements are there with regards to the dataset and the model?

And in conclusion the study has found that the CORD-19 dataset is not that well structured for training a transformer network from scratch, with regards to the diversification in data and the complexity of the language. The signs are there that it's learning, but it's not learning well enough to be used as a tool and would need to be trained on other datasets beforehand, or gain access to more data and be exposed to heavier preprocessing of the data. Once a sufficient set of samples are obtained and the validation shows heavier progress, additional quality controls can be implemented using

for example fine tuning via ROUGE-scores which is a common tool for scoring the overlap between actual summaries and generated ones.

The process of building the transformer from scratch was nevertheless a very important lesson, as many insights can be taken from the knowledge, with the most specific points being probably the relations between num_heads, d_model and the mechanisms behind attention. The relation between the d_model and num_heads is the crucial key for enabling the model to understand the context of the sentences it generates, when the data is complex in patterns akin to the CORD-19 dataset, there is a good argument to be made for increasing these parameters in scope. The simpler the data to understand, the less need for more features to encapsulate the relations. Under better circumstances, and increase in layers, d_model and num_heads would be preferable, together with a larger batch_size of at least around 32 and preferably 64, to improve the stability of the learning process.

## 7.2 Educational Highlights

A large focus for the study has also been on the aspect of learning how to work with a transformer model, how to build it from scratch and how to work with an open source unstructured and raw dataset. With these many challenges in mind, there are some key educational highlights I as the author want to share, as it might prove valuable to future researchers on the topic, if they seek to further develop a specific area of interest.

The strongest educational takeaway most definitely lies in understanding the concepts of attention in the transformer and how the mathematical principles correlate to each other and how they then translate into a very “human-like” understanding of learning. The more I worked with the model and the technology, I realized this is in essence exactly how we as people understand and communicate, but it's a mathematical approach to modeling it. This was an interesting insight, but also valuable, as it has given new ideas of how to continue working and improving on the model, but also sparked a flame of interest for continuing research into the transformer model.

Another highlight must certainly be to work on a raw and unstructured dataset like the CORD-19 dataset. There is a lot of problematic and tricky data in the dataset, but that in itself is rarely a bad case for a study such as this, where a large focus is on doing research and learning from data and the technology. The educational experience of working on the dataset gave a lot in terms of learning to find patterns in data, processing techniques and how to visualize large amounts of data.

Both the experiences of the transformer model and the CORD-19 dataset is something that I as the author have use of today in my work, separate from the thesis, and will surely continue to have use of in the future.

## 7.3 Future work

With regards to the findings of this study there are a couple of interesting subjects to delve deeper on. The closest in design option would be to temporarily swap datasets to try out the architecture on the CNN/Daily mail dataset, to evaluate its performance on simpler and more uniform data. This would give a good indication of the models total understanding of interpreting better prepared data.

Another approach would be to also try tuning the parameters of the project with the aim of finding a model better suited for the task and upscaling the computational restraints on the project. With the more computational efficiency at hand, more data could also be introduced and one could perhaps split the data into different sets, for example:

1. A set of 20000 papers for short summarization (short abstracts, short titles)
2. A set of 20000 papers for medium short summarization
3. A set of 20000 papers for longer summarization

This would be an interesting stepping stone, as the data presented in each breakpoint is more uniform, making it easier for the model to pick up on the patterns and fine tune it.

The final and last suggestion would be that of implementing extractive summarization and attempting summarization on the large text bodies into abstracts.

Extractive summarization could be used to take out the most important features of chunks in the longer texts, which would then act as the targets for the abstractive summarization, enabling a hierarchical approach as mentioned previously.

Overall, the topic of the transformer and summarization is a topic filled with mathematical opportunities, challenges and possibilities. One that definitely would need more research, and can provide some amazing solutions.

# 8. References


Allen Institute for AI. 2020. “COVID-19 Open Research Dataset Challenge (CORD-19).” Kaggle. https://www.kaggle.com/datasets/allen-institute-for-ai/CORD-19-research-challenge.

Amidi, Afshine, and Shervine Amidi. n.d. “Recurrent Neural Networks cheatsheet.” stanford. Accessed August 8, 2023. https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks.

Brown, Tom B. 2020. “Language Models are Few-Shot Learners.” arXiv. https://arxiv.org/abs/2005.14165.

Brownlee, Jason. 2022. “Why Initialize a Neural Network with Random Weights? - MachineLearningMastery.com.” Machine Learning Mastery. https://machinelearningmastery.com/why-initialize-a-neural-network-with-random-weights/.

Chiusano, Fabio. 2022. “Two minutes NLP — Learn the ROUGE metric by examples.” Medium. https://medium.com/nlplanet/two-minutes-nlp-learn-the-rouge-metric-by-examples-f179cc285499.

Culurciello, Eugenio. 2018. “The fall of RNN / LSTM. We fell for Recurrent neural networks… | by Eugenio Culurciello.” Towards Data Science. https://towardsdatascience.com/the-fall-of-rnn-lstm-2d1594c74ce0.

“Deep learning.” 2023. Wikipedia. https://en.wikipedia.org/wiki/Deep_learning.

Giacaglia, Giuliano. 2019. “How Transformers Work. Transformers are a type of neural… | by Giuliano Giacaglia.” Towards Data Science. https://towardsdatascience.com/transformers-141e32e69591.

Giacaglia, Giuliano. n.d. “How Transformers Work.” Towards Data Science. Accessed August 8, 2023. https://towardsdatascience.com/transformers-141e32e69591.

Hugging Face. n.d. “BART.” https://huggingface.co/docs/transformers/model_doc/bart.

Hugging Face. n.d. “cnn_dailymail · Datasets at Hugging Face.” Hugging Face. Accessed May 5, 2024. https://huggingface.co/datasets/cnn_dailymail.

IBM. n.d. “What is Natural Language Processing?” IBM. Accessed March 3, 2024. https://www.ibm.com/topics/natural-language-processing.

Khatri, Chandra, Gyanit Singh, and Nish Parikh. 2018. “Abstractive and Extractive Text Summarization using Document Context Vector and Recurrent Neural Networks.” arXiv. https://arxiv.org/abs/1807.08000.

“Logistic regression.” 2023. Wikipedia. https://en.wikipedia.org/wiki/Logistic_regression.

“Machine learning.” 2023. Wikipedia. https://en.wikipedia.org/wiki/Machine_learning.

Margani, Rashmi. 2019. “Comprehensive Hands-on Guide to Sequence Model batching strategy: Bucketing technique | by Rashmi Margani | Medium.” Rashmi Margani. https://medium.com/@rashmi.margani/how-to-speed-up-the-training-of-the-sequence-model-using-bucketing-techniques-9e302b0fd976.

Nallapati, Ramesh, Feifei Zhai, and Bowen Zhou. 2017. “SummaRuNNer: A Recurrent Neural Network Based Sequence Model for Extractive Summarization of Documents | Proceedings of the AAAI Conference on Artificial Intelligence.” https://ojs.aaai.org/index.php/AAAI/article/view/10958.

OpenAI. n.d. “openai/tiktoken: tiktoken is a fast BPE tokeniser for use with OpenAI's models.” GitHub. Accessed March 29, 2024. https://github.com/openai/tiktoken.

Phillips, Hunter. 2023. “Positional Encoding. This article is the second in The… | by Hunter Phillips.” Medium. https://medium.com/@hunter-j-phillips/positional-encoding-7a93db4109e6.

Sutskever, Ilya, Oriol Vinalys, and Quoc Le. 2014. “[1409.3215] Sequence to Sequence Learning with Neural Networks.” arXiv. https://arxiv.org/abs/1409.3215.

Sweta. 2020. “Batch , Mini Batch and Stochastic gradient descent | by Sweta | Medium.” Sweta. https://sweta-nit.medium.com/batch-mini-batch-and-stochastic-gradient-descent-e9bc4cacd461.

Taylor, Petroc. 2022. “Total data volume worldwide 2010-2025.” Statista. https://www.statista.com/statistics/871513/worldwide-data-created/.

Tensorflow. 2024. “Neural machine translation with a Transformer and Keras | Text.” TensorFlow. https://www.tensorflow.org/text/tutorials/transformer#setup_input_pipeline.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. “Attention Is All You Need.” arXiv. https://arxiv.org/abs/1706.03762.

Yadav, Saurabh. 2018. “Weight Initialization Techniques in Neural Networks | by Saurabh Yadav.” Towards Data Science.

https://towardsdatascience.com/weight-initialization-techniques-in-neural-networks-26c649eb3b78.

Yang, Charles. 2019. “Deep Learning in Science. A survey of opportunities and trends.” Towards Data Science. https://towardsdatascience.com/deep-learning-in-science-fd614bb3f3ce.

Zhu, Scott, and Francois Chollet. 2023. “Working with RNNs.” TensorFlow. https://www.tensorflow.org/guide/keras/rnn.